

Maha E-Seva Kendra

Production architecture, code structure, data model, SEO, performance, security and implementation rules

Purpose: give Antigravity one technical source of truth for implementing the website without inventing architecture during development.

Decision	Recommended implementation
Frontend	React + TypeScript + Vite
Styling	CSS architecture using design tokens; Tailwind optional only if it improves consistency
Routing	React Router with clean, SEO-friendly paths
Content	Data-driven service/scheme/blog objects; CMS/backend can be introduced later
Backend	Prefer a lightweight API only where dynamic management is required; static-first pages should not depend on an API
Deployment	Vercel or equivalent edge/static hosting for frontend
Source control	GitHub with feature branches and phase-based commits
Primary principle	Static-first, fast, accessible, data-driven, SEO-friendly

1. Architecture Goals

- 1 Build a fast public-facing website for a real Maha E-Seva Kendra in Nerul.
- 2 Represent all 32 services through reusable data and reusable components.
- 3 Make price, documents, process and contact actions easy to discover.
- 4 Support frequently changing government schemes without redesigning the UI.
- 5 Create a blog architecture capable of generating long-tail local SEO traffic.
- 6 Keep the initial experience fast even on mobile networks.
- 7 Prevent third-party services from blocking the critical rendering path.
- 8 Make content maintainable without duplicating 32 page implementations.
- 9 Allow future addition of a CMS/admin layer without rewriting the frontend.
- 10 Maintain a clear separation between verified business information, government information and editorial content.

2. Recommended Stack

Layer	Choice	Reason
UI	React + TypeScript	Reusable component system and strong maintainability.
Build	Vite	Fast local development and optimized production bundling.
Routing	React Router	Clean routes, nested service pages and 404 handling.
Styling	CSS Modules / scoped CSS or organized global CSS	Precise design system and low runtime overhead.
Icons	Lucide React or equivalent	Lightweight consistent SVG iconography.
Animation	CSS transitions + small motion utility / Framer Motion only where justified	Motion without turning every interaction into a heavy dependency.
Forms	Native form controls + lightweight validation	Fast and accessible.
Content	Typed local JSON/TS data initially	No CMS complexity required for launch.
Optional backend	Django/DRF or lightweight API if admin/content management is required later	Future extensibility.
Analytics	Privacy-conscious analytics / GA4 if required	Measure search, contact and service intent.
Hosting	Vercel / equivalent	CDN, HTTPS, preview deployments and easy rollback.

3. High-Level Architecture

Use a **static-first presentation architecture**. The public website should render core business content without waiting for a database or third-party API. Dynamic features are progressively enhanced.

Area	Architecture
Public pages	React routes → reusable templates → typed content/data
Service catalogue	Service data → category/filter logic → ServiceCard → ServiceDetailTemplate
Schemes	Scheme data → SchemeCard → SchemeDetailTemplate
Blog	Article metadata/content → BlogCard → ArticleTemplate
Forms	Client validation → secure form endpoint/provider → success/error state
Maps	Static/lightweight location CTA → external map only on intent
WhatsApp	Direct deep link with optional prefilled contextual message
SEO	Route-level metadata + structured data + sitemap + robots

Area	Architecture
Assets	Optimized local assets/CDN; responsive images
Analytics	Event abstraction so provider can be changed later

4. Project Folder Structure

Use a feature-oriented structure rather than a giant components folder.

Recommended structure

```
src/
  app/
    router/
    providers/
    app.tsx
    components/
    ui/
    layout/
    navigation/
    service/
    scheme/
    blog/
    contact/
    seo/
    pages/
    Home/
    Services/
    ServiceDetail/
    Schemes/
    SchemeDetail/
    Blog/
    Article/
    About/
    Contact/
    NotFound/
    data/
    services.ts
    categories.ts
    schemes.ts
    faqs.ts
    business.ts
    lib/
    seo/
    analytics/
    validation/
    utils/
    hooks/
    styles/
    tokens.css
    globals.css
    utilities.css
    assets/
    types/
```

Do not create one custom page component per service. All service pages must use the same template and service data.

5. Route Architecture

Route	Purpose
/	Homepage
/services	All 32 services
/services/:slug	Individual service
/schemes	Current government schemes
/schemes/:slug	Scheme detail

Route	Purpose
/blog	Blog index
/blog/:slug	Article
/about	About the centre
/contact	Contact + location
/faq	Frequently asked questions
/404	Not found

URL rules

- Use lowercase, human-readable slugs.
- Do not use IDs in public URLs.
- Do not use query strings for canonical service pages.
- Use hyphens, not underscores.
- Keep slugs stable once indexed.
- If a slug changes, implement a permanent redirect.
- Canonical URLs must be generated consistently.

6. Core TypeScript Data Model

The following models are the contract between content and UI.

Service

id: string
slug: string
name: string
category: ServiceCategory
shortDescription: string
description: string
price: number | null
priceLabel: string
governmentFee: string | null
documents: DocumentRequirement[]
eligibility: string[]
processSteps: ProcessStep[]
processingInfo: string | null
faqs: FAQ[]
popular: boolean
featured: boolean
status: 'active' | 'paused'
lastUpdated: string
seo: SEOData

DocumentRequirement

name: string
description?: string
required: boolean
note?: string

Scheme

id, slug, title, status, summary, overview, eligibility, benefits, documents, applicationSteps, dates, officialSource, lastUpdated, relatedServices, seo

Article

id, slug, title, excerpt, content, category, author, publishedAt, updatedAt, featuredImage, readingTime, relatedServices, relatedArticles, seo

Important: pricing and document requirements must be treated as business content, not UI constants. This prevents duplicated information and allows later editing from one source.

7. Initial 32-Service Data Inventory

#	Service	Category	Known centre price
1	Aadhaar Smart Card	Identity	■40
2	PAN Card	Identity	■150
3	Voter ID	Identity	■100
4	Passport Online Application	Travel / Government	■300 + ■1,500 government fee
5	Driving Licence	Transport	■300
6	ABHA Health Card	Health	■50
7	Ayushman Bharat Card	Health	■50
8	Senior Citizen Card	Certificates / Welfare	■100
9	E-Shram Card	Employment / Welfare	■50
10	UAN PF Card	Employment / Banking	■100
11	Domicile Certificate	Certificates	■300
12	Income Certificate	Certificates	■300
13	Residence Certificate	Certificates	■300
14	Non-Creamy Layer	Certificates	■700
15	Caste Certificate	Certificates	■300
16	Police Verification	Verification	■500
17	E-Registration Rent	Business / Registration	■1,000
18	Lost Aadhaar / PAN Card	Identity	■100 / ■200
19	Small Scale Business (MSME) License	Business	■500
20	Shop Establishment / Gumasta License	Business	■500
21	Name Change e-Gazette	Government / Legal	■1,200
22	Banking Related Services	Banking	■100
23	Health Insurance	Insurance	■100
24	Motor Insurance	Insurance	■100
25	Digital Life Certificate for Pensioners	Pension	■200
26	Passport Size Photos	Photography / Printing	■100 / 30 photos
27	A4 & Card Lamination	Printing	■10 / 5
28	Money Transfer & Withdrawal	Banking	Not provided
29	Digital Album Photo Quality of Important Documents	Digital / Printing	Not provided
30	Government Job & Scheme Online Form Filling	Forms	Not provided
31	All Types Bank Loan	Banking	Not provided
32	Insurance Services	Insurance	Not provided

Pricing accuracy rule

The source business data does not provide confirmed prices for services 28–32. Do not invent prices. Render 'Contact for pricing' or an equivalent verified label until the owner supplies the charge. Government fees must never be assumed from the centre service charge.

8. Service Component Architecture

Component	Responsibility
ServiceCard	Compact service discovery card.

Component	Responsibility
ServiceGrid	Responsive grid, filtering and search.
ServiceFilters	Category/search controls.
ServiceHero	Service title, price and primary CTA.
PriceBreakdown	Centre fee vs government fee.
DocumentList	Requirements with notes.
ProcessSteps	Numbered process.
ServiceFAQ	Service-specific questions.
RelatedServices	Contextual cross-links.
ServiceCTA	Call/WhatsApp/visit actions.
ServiceDetailTemplate	Composes all service sections.

9. Homepage Component Architecture

HomePage

→ AnnouncementBar
 → Header
 → HeroSection
 → TrustHighlights
 → PopularServices
 → ServicesPreview
 → SchemesPreview
 → WhyChooseUs
 → PricingHighlights
 → ReviewsSection
 → BlogPreview
 → LocationSection
 → FAQPreview
 → FinalCTA
 → Footer

Every section must be independently reusable and hideable when content is absent. Do not leave empty visual gaps when a dynamic section has no data.

10. SEO Architecture

SEO must be implemented at the architecture level, not added after visual development.

Requirement	Implementation
Title	Unique route-level title; service and local intent where natural.
Meta description	Unique, useful summary; avoid keyword stuffing.
Canonical	One canonical URL per indexable page.
Open Graph	Title, description and optimized social image.
Robots	Allow public content; block accidental admin/private routes.
Sitemap	Generate/update for services, schemes, blog and key pages.
Breadcrumbs	Visible and structured where appropriate.
Structured data	LocalBusiness/appropriate business schema, BreadcrumbList, Article and FAQ where eligible and accurate.
Internal links	Service ↔ related service ↔ blog ↔ scheme.
Heading structure	One clear H1, logical H2/H3 hierarchy.
Images	Descriptive alt text; decorative images use empty alt.
Local SEO	Nerul/Navi Mumbai context naturally integrated into business and service content.

SEO route examples

`/services/domicile-certificate` — Domicile Certificate Assistance in Nerul

`/services/pan-card` — PAN Card Application Assistance in Nerul

`/blog/how-to-apply-for-domicile-certificate-in-maharashtra` — informational long-tail content

Do not create dozens of near-duplicate pages simply by swapping neighbourhood names. Local pages should exist only when they provide genuinely useful unique content.

11. Local SEO Architecture

- Maintain one canonical business identity across website and Google Business Profile.
- Display the centre's real address, phone numbers and business hours consistently.
- Create an About/Contact page with clear location context.
- Use natural phrases such as Maha E-Seva Kendra, Nerul, Navi Mumbai and Maharashtra where relevant.
- Create genuinely useful service pages for high-intent searches.
- Publish local informational articles that answer real customer questions.
- Add directions CTA and location information without embedding a heavy map by default.
- Use structured business data only for facts that are actually true.

Business data currently supplied: Shop No-15, Janta Market Bridge, Nerul East, Sector 3, Nerul, Navi Mumbai, Maharashtra 400706; phone 099877 72424 and 8850 77 24 24. Verify the final public business details with the owner before launch.

12. Blog Technical Architecture

Start with local typed content files or Markdown/MDX-style content if the team is comfortable with it. The content layer must make it easy to publish without touching UI components.

Recommended article front matter: slug, title, excerpt, category, author, publishedAt, updatedAt, featuredImage, readingTime, relatedServices, relatedArticles, canonical, noindex and SEO description.

Blog categories can include: Certificates, Aadhaar & PAN, Passport, Government Schemes, Banking, Insurance, Business Services, Maharashtra Guides and Local Guides.

13. Schemes Technical Architecture

Schemes change frequently. Keep scheme content independent from service data.

- Each scheme has a status and lastUpdated value.
- Closed schemes remain accessible only if they have lasting informational value; otherwise archive/noindex according to editorial policy.
- Never present an old scheme as currently open.
- Where possible, link to the official government source.
- Create a simple owner/editor workflow later if manual content updates become frequent.

14. Forms & Backend Boundary

The initial website does not require a complex backend for the public catalogue. Forms are the main dynamic boundary.

Feature	Initial approach	Future upgrade
Contact enquiry	Secure serverless/API form endpoint	CRM/database
WhatsApp	Direct external link	CRM attribution if needed
Blog	Static content	Headless CMS

Feature	Initial approach	Future upgrade
Schemes	Static typed data	Admin/CMS
Service data	Typed data	Admin/CMS
Analytics	Client analytics abstraction	Advanced reporting

15. Contact Form Security

- Validate fields on both client and server.
- Rate-limit submissions.
- Do not accept sensitive identity documents through the basic public form.
- Do not store unnecessary personal data.
- Protect against spam with honeypot/rate limiting or a privacy-conscious challenge where necessary.
- Escape/sanitize any user-submitted content before displaying it.
- Do not expose private email addresses in frontend source when avoidable.

16. WhatsApp Integration

Use a normal WhatsApp deep link. The website should not require a WhatsApp SDK.

Suggested contextual message pattern: 'Hello Maha E-Seva Kendra, I need help with [Service Name]. Please tell me the documents and process.'

For a generic CTA, use a neutral message. Do not automatically include Aadhaar, PAN or other sensitive identifiers.

17. Performance Architecture

Performance is a launch requirement, not an optimization phase.

Area	Rule
Initial JS	Keep route-critical JavaScript small.
Images	WebP/AVIF where supported; responsive sizes; explicit dimensions.
Fonts	Use a minimal font set; avoid loading unnecessary weights.
Icons	Inline/lazy SVG or lightweight icon package.
Third-party	Load analytics, maps and embeds without blocking critical render.
Map	Click-to-load or external directions link.
Animations	CSS transform/opacity where possible; avoid layout-triggering animation.
Content	Render core text/content immediately.
Caching	Use CDN/static asset caching.
CLS	Reserve image/embedded content dimensions.
Mobile	Test on throttled network and mid-range Android devices.

18. Core Web Vitals Targets

Use these as engineering targets rather than promises. The goal is consistently good real-user performance.

Metric	Target
LCP	≤ 2.5s
INP	≤ 200ms

Metric	Target
CLS	≤ 0.1
Initial page weight	Keep as low as practical; avoid unnecessary libraries/assets.
Lighthouse	Aim for strong Performance/SEO/Accessibility scores, validated on production build.

19. Image Strategy

- Use real shop/business photos when available and approved.
- Compress all uploads before committing.
- Provide width/height attributes or aspect-ratio containers.
- Use srcset/sizes for responsive images.
- Lazy-load below-the-fold images.
- Do not use large original Google Maps images or screenshots as page backgrounds.
- Keep hero media optimized and visually stable.

20. Animation Engineering

- Prefer transform and opacity.
- Use IntersectionObserver or an equivalent lightweight mechanism for scroll reveals.
- Only animate elements when they enter the viewport.
- Do not animate all cards individually with large stagger delays.
- Respect prefers-reduced-motion.
- Keep animation code centralized in small reusable utilities.
- Remove animation dependencies that do not justify their bundle cost.

21. Accessibility Engineering

- Use semantic HTML before ARIA.
- Buttons must be buttons; links must be links.
- Service cards should use a real link to service pages.
- Accordion headers should expose expanded state.
- Form labels must be associated with inputs.
- Errors should be announced appropriately.
- Keyboard focus must remain visible.
- Do not trap focus except in a true modal/dialog.
- Check colour contrast and reduced motion.

22. State Management

Do not introduce Redux or another global state system unless actual application state requires it. Most public content is immutable data. Local component state is sufficient for filters, accordions, mobile navigation and forms.

If an API/CMS is introduced later, use a dedicated server-state/data-fetching layer only where it provides measurable value.

23. Error & Fallback Strategy

- Unknown service slug → 404 with Services CTA.
- Missing service data → development error + production graceful fallback.
- Broken image → alt text / fallback asset.
- Form endpoint unavailable → call/WhatsApp fallback.
- External map unavailable → address + directions link still works.
- Analytics unavailable → website remains fully functional.
- Scheme source unavailable → do not claim live verification; show last verified update.

24. Analytics Event Architecture

Create a small analytics abstraction so UI components do not depend directly on a vendor.

Event	Parameters
service_view	serviceSlug, category
service_cta_click	serviceSlug, action
whatsapp_click	serviceSlug?, location
call_click	serviceSlug?, location
directions_click	location
contact_submit	service?, success/failure
scheme_view	schemeSlug
blog_view	articleSlug
service_search	query
service_filter	category

25. Security & Privacy

- Serve the site over HTTPS.
- Keep secrets out of the repository and client bundle.
- Use environment variables for server-side credentials.
- Do not put API keys in public source unless the provider explicitly requires a public key.
- Minimize personal-data collection.
- Do not upload or retain user identity documents unless a future feature explicitly requires it and appropriate security/privacy controls exist.
- Set sensible security headers where hosting permits.
- Keep dependencies updated and review vulnerabilities before production.

26. SEO Content/Data Separation

Business facts must be separated from editorial copy. For example, the phone number, address, price and business hours should come from a central business object. Service descriptions, blog posts and scheme explanations should be content objects.

This prevents accidental inconsistencies such as different phone numbers or prices appearing on different pages.

27. Business Configuration Object

Business
name

legal/displayName
shortDescription
address
phoneNumbers[]
hours[]
googleMapsUrl
whatsappNumber
ratingDisplay? (only if verified/current)
socialLinks[]
serviceArea[]
lastVerified

Owner-facing information should be verified before launch. The supplied Google Maps URL can be used as the directions/business-profile reference, but the website should not scrape Google Maps dynamically.

28. Development Rules for Antigravity

- 1 Read PRD #1 and Design/UX #2 before writing implementation code.
- 2 Implement the design tokens and global layout before individual pages.
- 3 Build reusable primitives first: Button, Card, Badge, Container, SectionHeading, IconButton, Input, Accordion.
- 4 Build ServiceCard and ServiceDetailTemplate before adding the 32 service records.
- 5 Add all service records through data, not duplicated JSX.
- 6 Build one blog template and one scheme template.
- 7 Implement SEO metadata centrally.
- 8 Implement responsive behavior before polishing desktop animation.
- 9 Run production builds regularly; do not wait until the end to discover bundle problems.
- 10 Do not introduce libraries for problems that can be solved with small native/CSS utilities.
- 11 Do not alter the approved visual direction without a documented reason.
- 12 Do not invent business facts, prices, documents, government fees or scheme eligibility.

29. Recommended Implementation Phases

Phase	Deliverable	Exit condition
1	Foundation	React/Vite setup, tokens, fonts, global CSS, routing, layout.
2	UI primitives	Buttons, cards, inputs, badges, accordions, containers.
3	Navigation + homepage shell	Header, footer, hero, CTA, responsive layout.
4	Service engine	Data model + all 32 records + grid + filters + detail template.
5	Schemes	Listing/detail + freshness/status system.
6	Blog	Listing/detail + content structure + related links.
7	Contact/location	Form, WhatsApp, phone, directions, hours.
8	SEO	Metadata, schema, sitemap, robots, canonical, internal linking.
9	Performance/accessibility	Production audit, image optimization, reduced motion, keyboard QA.
10	Final content/launch	Business verification, analytics, redirects, production QA.

30. Testing Strategy

Test	Minimum expectation
Component	Critical reusable UI components render correctly.

Test	Minimum expectation
Route	Every public route loads and has correct metadata.
Responsive	Mobile, tablet and desktop layouts checked.
Keyboard	Navigation, forms and accordions usable without mouse.
SEO	Titles, descriptions, canonical, sitemap and structured data checked.
Performance	Production build tested with throttled mobile network.
Content	All 32 services checked against source business data.
Links	Phone, WhatsApp, directions, internal/external links verified.
Forms	Validation, success and failure paths verified.
404	Invalid routes return useful recovery UI.

31. Deployment & Git Workflow

- main = production-ready branch.
- Feature branches should correspond to implementation phases or focused features.
- Every phase should produce a working build.
- Use pull requests or review checkpoints even for solo development where practical.
- Never commit secrets, .env files or private credentials.
- Use preview deployments for UI review.
- Production environment variables must be configured separately.
- Maintain a rollback-capable production deployment.

32. Definition of Done

The website is technically complete only when:

- All 32 services exist in the central data model.
- Every service has a stable URL and uses the same detail template.
- Prices match verified owner information.
- Unknown pricing is explicitly marked rather than invented.
- Document requirements are verified before publication.
- Schemes have freshness/update information.
- Blog pages have unique metadata and internal links.
- Core pages have canonical URLs and sitemap inclusion.
- Contact, call, WhatsApp and directions work on mobile.
- Production build is fast on mobile.
- Keyboard and reduced-motion behavior work.
- 404 and error states are useful.
- No major console errors or broken links remain.
- Owner verifies address, phone, hours, pricing and public claims before launch.

33. Technical Non-Negotiables

1. No duplicated 32-page JSX implementations.
2. No invented business information.
3. No heavy third-party map/video on the critical path.
4. No animation that delays content or harms accessibility.
5. No SEO generated by copying the same text across pages.
6. No sensitive document collection through a basic contact form.
7. No hard-coded service prices scattered throughout components.
8. No dependency on an API for content that can be statically delivered.
9. No launch until real business facts are owner-verified.
10. Every phase must leave the application in a runnable state.

34. Final Architecture Principle

Build the website as a small, fast publishing platform—not as a collection of screens. Services, schemes and articles are content. Components are reusable presentation. SEO is generated from the content model. Performance is protected by the architecture. This allows the centre to keep adding services, schemes and useful local information without repeatedly redesigning the website.